



Razvoj mobilnih sistema i servisa

- Konkurentnost, servisi i background zadaci -

**Katedra za računarstvo
Elektronski fakultet u Nišu**



Literatura

- ✿ G. Meike, *Android Concurrency (Android Deep Dive)*, Addison-Wesley Professional; 1st edition (June 20, 2016)
- ✿ Andrés Ibañez Kautsch, *Modern Concurrency on Apple Platforms: Using async/await with Swift*, Apress; 1st ed. edition (November 15, 2022)
- ✿ Donny Wals, *Practical Swift Concurrency*, <https://practicalswiftconcurrency.com/>

Procesi i niti u razvoju softvera

- ❖ Interaktivni sistemi obrađuju događaje koje je korisnik izazvao interakcijom sa elementima korisničkog interfejsa
- ❖ Najčešće postoji red objekata koji predstavljaju generisane događaje i koji čekaju da budu obrađeni (message queue)
- ❖ Najčešće postoji jedna petlja u kojoj se iz reda preuzimaju objekti koji predstavljaju događaje i obrađuju (message pump/loop)
- ❖ Programski kod koji izvršava message loop se izvršava u glavnoj niti (UI niti)
 - ❖ Svaka funkcija koja obrađuje događaj mora brzo da se završi



Android – procesi i niti

- ✿ Kada se startuje neka komponenta aplikacije i ni jedna druga komponenta te aplikacije se ne izvršava, OS startuje novi Linux proces sa jednom niti
- ✿ Ukoliko već postoji neka komponenta te aplikacije koja se izvršava, nova komponenta će se izvršavati u istom procesu i glavnoj niti
- ✿ Moguće je definisati da se neke komponente aplikacije izvršavaju u posebnom procesu, a moguće je i kreirati nove (worker) niti

Android – procesi

- Podrazumevano, sve komponente jedne aplikacije se izvršavaju u jednom procesu
- U manifest-u je moguće definisati za svaki tip komponente (`activity`, `service`, `receiver` i `provider`) atribut `android:process`

```
<service  
    android:name=".MyService"  
    android:process=":my_service_process" />
```

- Android OS u nekom trenutku može uništiti neke procese kada ponestane memorije
- Uništavaju se prvenstveno procesi u kojima se izvršavaju `activity` koje nisu vidljive (u tom procesu se mogu izvršavati i još neke komponente)



Android – procesi

- Android OS rangira procese

- Foreground process**

 - Proces izvršava activity sa kojim korisnik interaguje

- Visible process**

 - Proces izvršava aktivnost koja je pauzirana (vidljiva ali korisnik ne interaguje sa njom)

- Service process**

 - Proces izvršava servis koji nije vezan za vidljiv activity

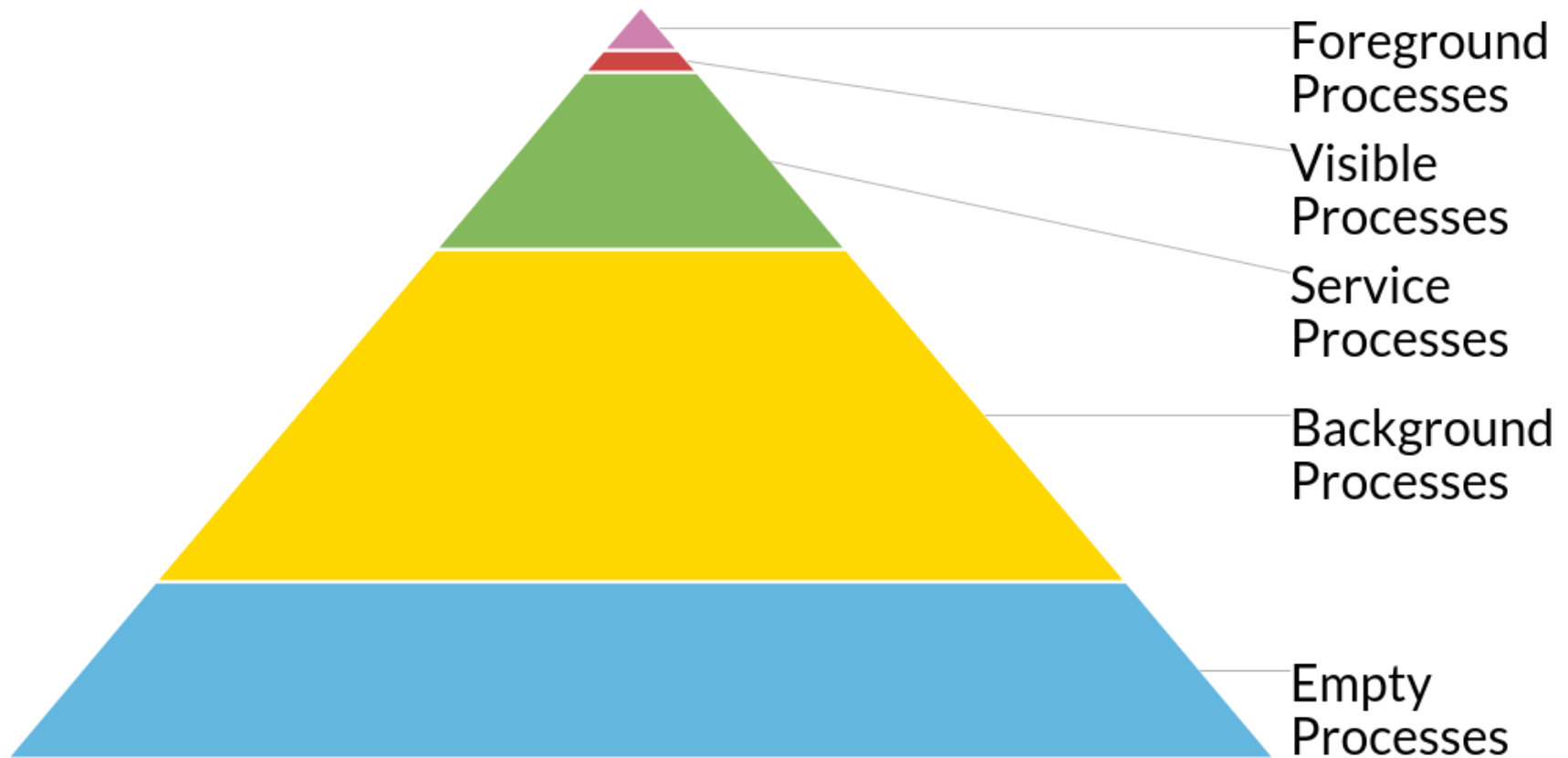
- Background process**

 - Proces izvršava aktivnosti koje nisu vidljive i koje su sortirane po LRU (Least Recently Used)

- Empty process** – kešira neaktivne komponente

Android – procesi

- Android OS može da uništi samo kompletan proces, ne pojedinačne komponente



Android – procesi

- ✿ Svaki proces se izvršava sa svojim jedinstvenim Linux user ID (UID) koji se dodeljuje aplikaciji prilikom instalacije
- ✿ Ovaj pristup izoluje podatke i izvršenje koda
- ✿ `adb shell ps`

```
USER      PID    PPID  VSIZE  RSS    PC    NAME
root              319    1      1537236 31324 S  zygote
...
u0_a221   5993   319    1731636 41504 S  com.whatsapp
u0_a96    3018   319    1640252 29540 S  com.dropbox.android
u0_a255   4892   319    1583828 34552 S  com.accuweather.android...
```

- ✿ Proces *Zygote* (inicijalna ćelija kada se novi organizam formira)
 - ▣ Parent svim Android procesima (kešira biblioteke)

Android – procesi

- Do API verzije 29 bilo je moguće koristiti shared UID

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="string"
    android:sharedUserId="string"
    android:sharedUserLabel="string resource"
    android:sharedUserMaxSdkVersion="integer"
    android:versionCode="integer"
    android:versionName="string"
    android:installLocation=["auto" | "internalOnly" | "preferExternal"] >
    ...
</manifest>
```

- ART (Android Runtime) ili Dalvik virtuelne mašine
- Android se trudi da deli stranice memorije za više procesa



Android – procesi

- ✚ Komunikacija između procesa (IPC – Inter Process Communication)
- ✚ Tradicionalne Linux metode
 - ✚ Pipes
 - ✚ Network sockets
 - ✚ Shared files
- ✚ Android specifične metode
 - ✚ Intent
 - ✚ Binder
 - ✚ Messenger (BroadcastReceiver)
- ✚ Android IPC omogućava kontrolu bezbednosti

Android – procesi

- ✱ **Intent-i** omogućavaju slanje podataka aktivnosti tako što se podaci pakuju u Extras deo Intent-a
- ✱ Funkcioniše i ako aktivnost koja se kreira se startuje u zasebnom procesu korišćenjem android:process atributa u manifest-u
- ✱ Mana – podaci se mogu preneti samo prilikom kreiranja procesa ili uništavanja (za startActivityForResult)

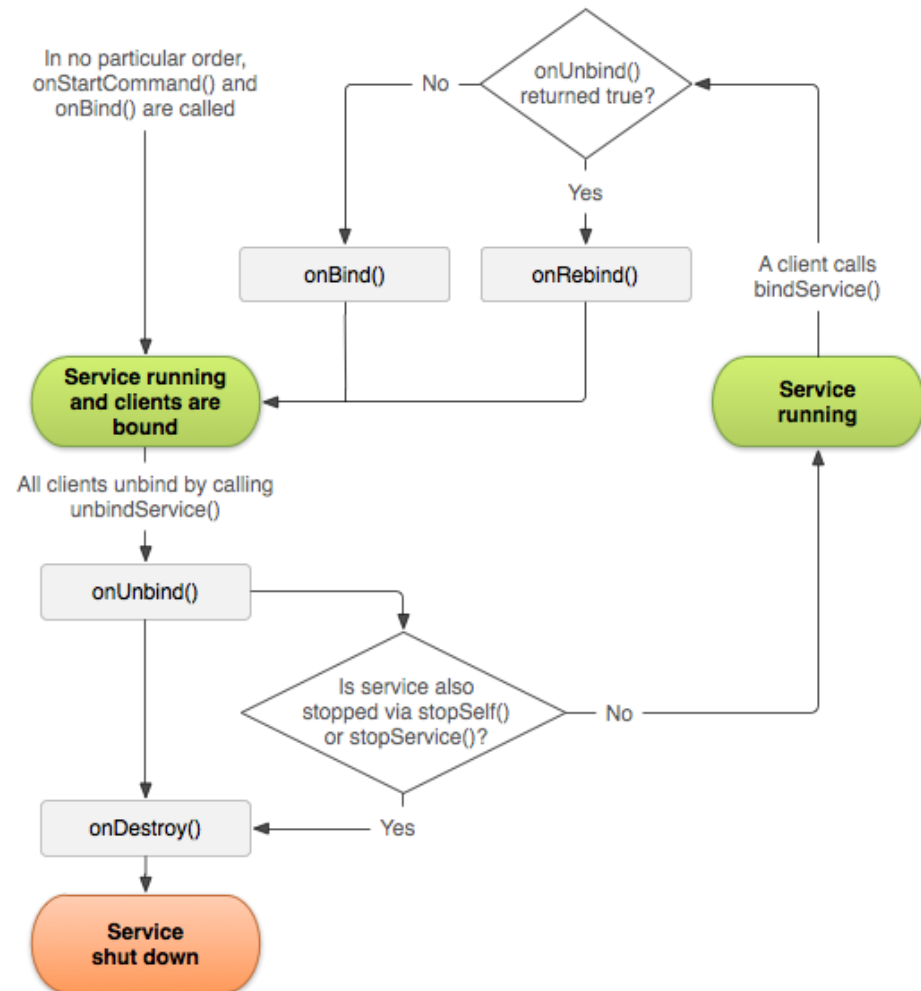
```
Intent intent = new Intent(this, SomeActivity.class);  
intent.putExtra("key", "some serialisable data");  
startActivity(intent); //or startActivityForResult(intent,Code) if we  
expect the invoked activity to response with some data
```

Android – procesi

- ✿ **Bound service** omogućava drugim aplikacijama da se povežu na servis i interaguju sa njim
- ✿ Implementacijom `onBind()` callback metode
- ✿ Ova metoda vraća `IBinder` interfejs kojim se definiše komunikacija
- ✿ Ovo je preferiran metod za IPC u okviru iste aplikacije
 - ✦ Jasno definisan interfejs
 - ✦ Sva komunikacija MORA da ide preko servisa i svi podaci moraju biti u skladu sa definisanim contract-om

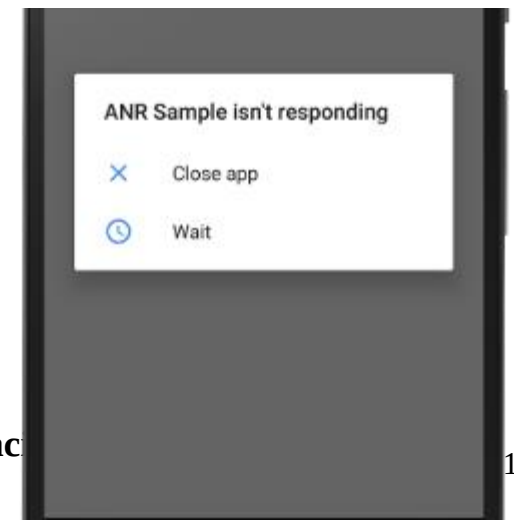
Android – procesi

Memorijski životni ciklus za bound service



Android – niti

- ✱ Kada se kreira proces kreira se i jedna nit, glavna nit (main thread)
- ✱ U ovoj niti aplikacija interaguje sa svim UI elementima (`android.widget` i `android.view`)
- ✱ Zato se glavna nit često naziva i UI nit
- ✱ OS **NE** kreira posebnu nit za svaku komponentu
- ✱ Svaka event-handling funkcija se izvršava u glavnoj niti
- ✱ Ako je glavna nit zauzeta duže od par (5) sekundi prikazuje se Application not responding (ANR)



Android – niti

- Android UI toolkit **NIJE** thread-safe
- Manipulisanje UI elementima nije moguće iz worker niti, samo iz UI niti
- Posledice Android single-thread modela
 - Ne blokirati UI nit
 - Ne manipulirati UI elementima iz worker niti
- Android nudi nekoliko jednostavnih načina kako iz worker niti pristupiti UI niti

`Activity.runOnUiThread(Runnable)`

`View.post(Runnable)`

`View.postDelayed(Runnable, long)`

```
fun onClick(v: View) {  
    Thread(Runnable {  
        // A potentially time consuming task.  
        val bitmap = processBitMap("image.png")  
        imageView.post {  
            imageView.setImageBitmap(bitmap)  
        }  
    }).start()  
}
```

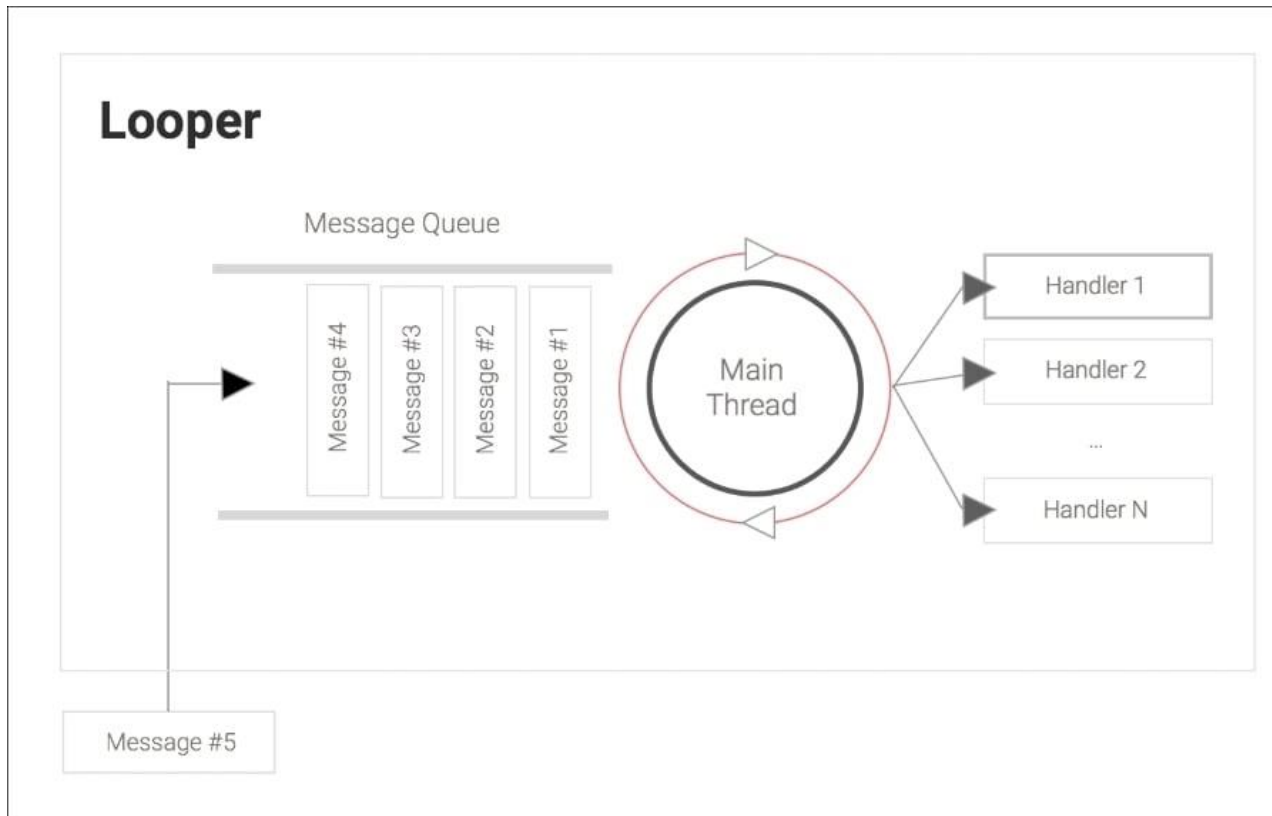
Android – niti

- ✱ Za razliku od procesa sve niti dele istu memorijsku oblast
 - ✘ Podaci između procesa se kontrolišu samo scope-om
- ✱ Spisak niti u DDMS-u

ID	Tid	Status	utime	stime	Name
1	30155	Runnable	4	8	main
*2	30161	Wait	0	0	Signal Catcher
*3	30162	Runnable	104	94	JDWP
*4	30166	Wait	0	0	FinalizerDaemon
*5	30165	Wait	0	0	ReferenceQueueDaemon
6	30164	Runnable	0	0	Binder_1
*7	30170	Monitor	0	0	GCDaemon
*8	30169	Wait	0	0	HeapTrimmerDaemon
9	30167	Runnable	0	2	Binder_2
*10	30168	Wait	0	0	FinalizerWatchdogDaemon
11	30198	Runnable	4	5	RenderThread

Android – niti

- ❖ Red poruka, Looper i Handler-i
- ❖ Glavna (UI) nit ima zakačen Looper koji obrađuje poruke





Android – niti

- ✿ Android specifičnosti vezane za niti
- ✿ Android 5.0 uvodi *RenderThread* kako bi UI animacije ostale glatke čak i kada je UI nit zauzeta
- ✿ Android pokušava da održi 60fps
 - ✦ Posledica – glavna nit ne bi trebalo da bude blokirana duže od **16ms**!
- ✿ Pomoćni Android feature-i
 - ✦ U Developer options postoji Strict mode – ekran blinka kada detektuje duge operacije u glavnoj niti
 - ✦ Od Honeycomb (API level 11) postoji `NetworkOnMainThreadException` koji se emituje kada sistem detektuje mrežnu aktivnost u glavnoj niti



Android – niti

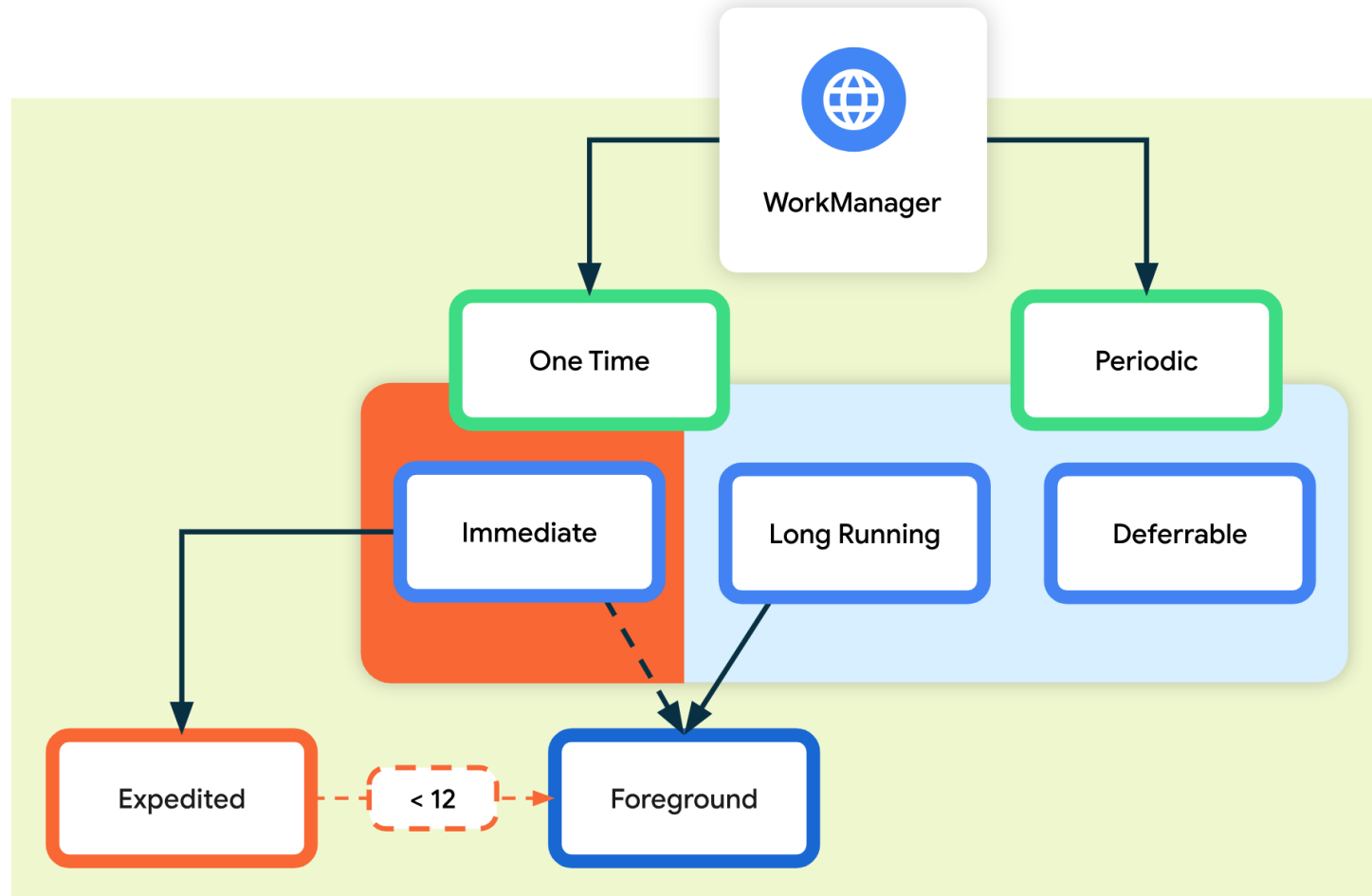
- ✚ Tipovi dugotrajnih zadataka
 - ✚ Asinhroni posao (Asynchronous Task)
 - ✚ Posao u pozadini (Background work)
 - ✚ Vidljiv servis (Foreground service)
- ✚ Opcije za Android konkurentno programiranje
 - ✚ Java.thread (*Looper, Handler, Runnable, Message*)
 - ✚ Thread wrapper-i (*AsyncTask, AsyncTaskLoader*)
 - ✚ Kotlin coroutines
 - ✚ WorkManager

Android – niti

- ✿ WorkManager je preporučen metod za implementaciju perzistentnih zadataka
- ✿ WorkManager obrađuje tri tipa perzistentnog posla
 - ✦ Trenutni – Mora da počne odmah i brzo da se završi
 - ✦ Dugotrajni – Može da traje dugo, potencijalno duže od 10min, može da bude prekinut
 - ✦ Odloženi – Zadatak koji može da se zakaže da počne kasnije i da se izvršava periodično

Android – niti

WorkManager – tipovi zadatka



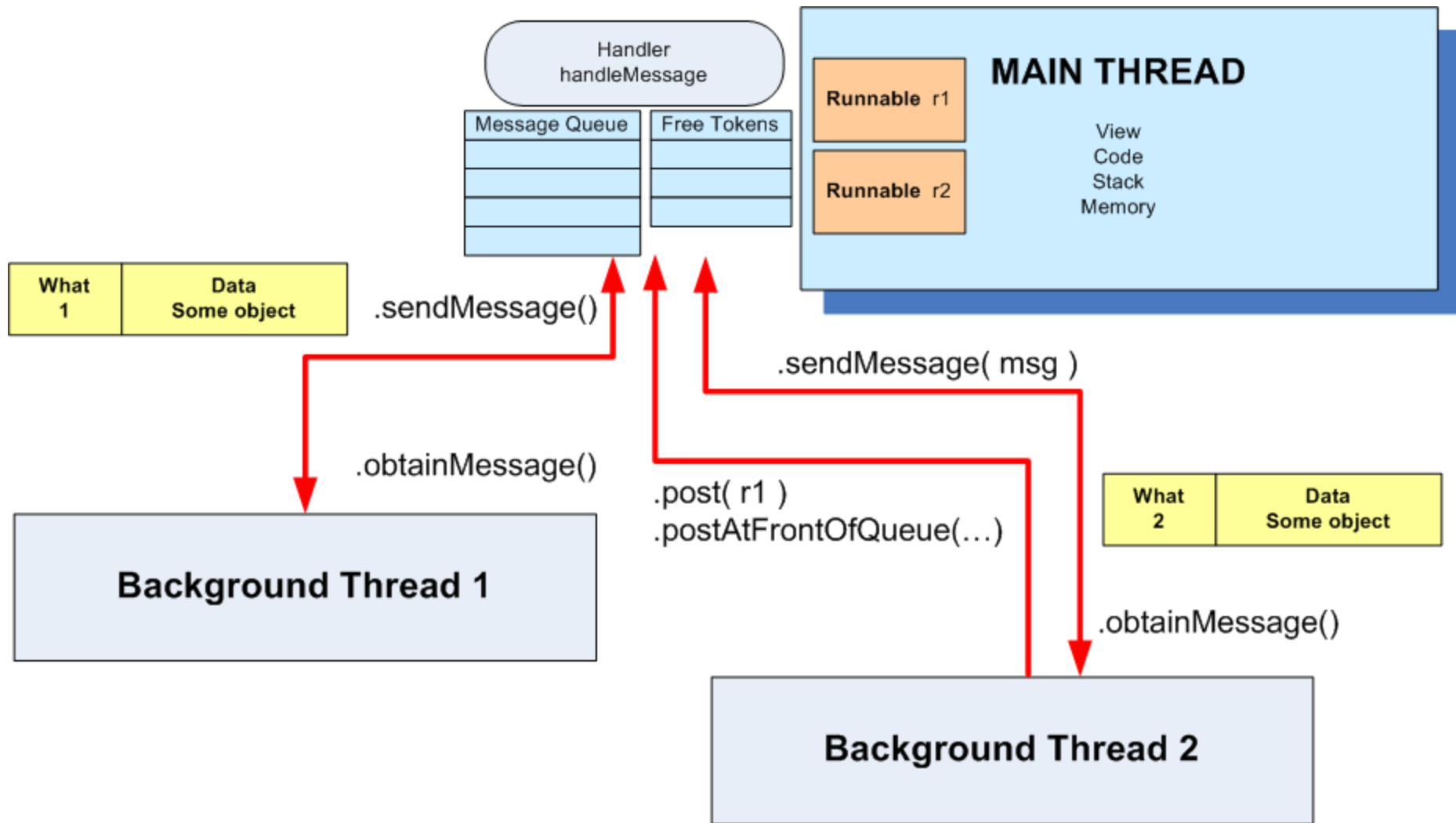
Android – niti

- ✚ Java niti – alternativne biblioteke
 - ✚ Guava
 - ✚ RxJava
- ✚ Kreiranje niti je skupa operacija koristiti thread pool koji se kreira odmah po startovanju aplikacije

```
public class MyApplication extends Application {  
    ExecutorService executorService = Executors.newFixedThreadPool(4);  
}
```

- ✚ Nasleđivanjem klase *Thread*
- ✚ Implementiranjem interfejsa *Runnable*
- ✚ Komunikacija između niti
 - ✚ Korišćenjem Message i Handler objekata
 - ✚ Prosleđivanjem Runnable objekata

Android – niti



Android – niti

Handle primer

Main Thread	Background Thread
<pre>... Handler myHandler = new Handler() { @Override public void handleMessage(Message msg) { // do something with the message... // update GUI if needed! ... } //handleMessage }; //myHandler ...</pre>	<pre>... Thread backgJob = new Thread (new Runnable () { @Override public void run() { //...do some busy work here ... //get a token to be added to //the main's message queue Message msg = myHandler.obtainMessage(); ... //deliver message to the //main's message-queue myHandler.sendMessage(msg); } //run }); //Thread //this call executes the parallel thread backgroundJob.start(); ...</pre>

Android – niti

Post primer

Main Thread	Background Thread
<pre>... Handler myHandler = new Handler(); @Override public void onCreate(Bundle savedInstanceState) { ... Thread myThread1 = new Thread(backgroundTask, "backAlias1"); myThread1.start(); } // onCreate ... // this is the foreground runnable private Runnable foregroundTask = new Runnable() { @Override public void run() { // work on the UI if needed } } ...</pre>	<pre>// this is the "Runnable" object // that executes the background thread private Runnable backgroundTask = new Runnable () { @Override public void run() { ... Do some background work here myHandler.post(foregroundTask); } // run }; // backgroundTask</pre>

Android – niti

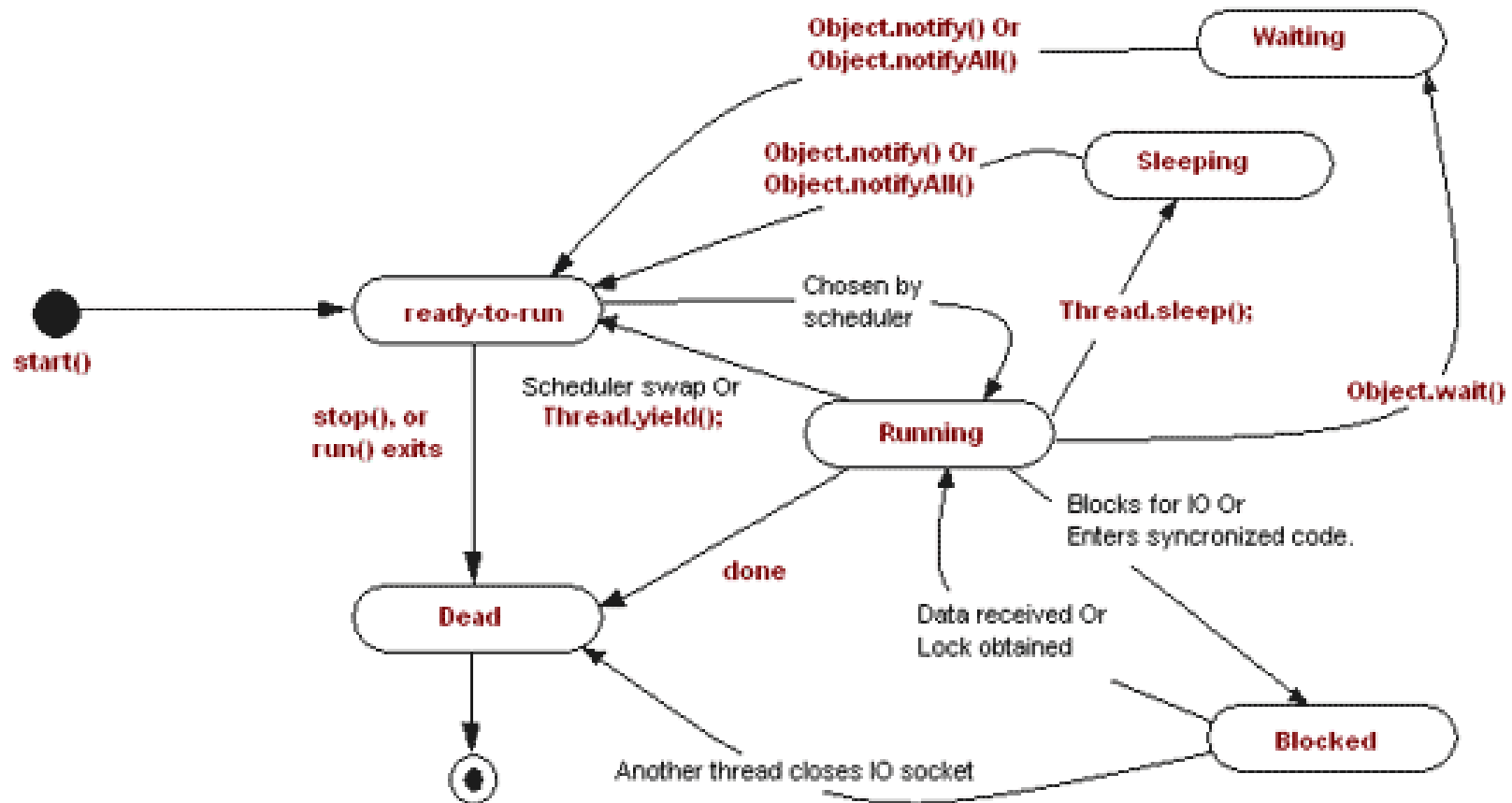
✚ Poruka se uzima od Handler-a sa

```
String localData = "Greeting from thread 1";  
Message mgs = myHandler.obtainMessage (1, localData);
```

- ✚ Poruke se vraćaju u red sa *sendMessage()*
 - ✚ **sendMessage()** – dodaje poruku na kraj reda
 - ✚ **sendMessageAtFrontOfQueue()** – ubacuje poruku na početak reda
 - ✚ **sendMessageAtTime()** – poslaće poruku u red u konkretno vreme na osnovu broja milisekundi u odnosu na *SystemClock.UptimeMillis()*
 - ✚ **sendMessageDelayed()** – poslaće poruku posle zadatog broja milisekundi

Android – niti

- Android thread-ovi imaju ista stanja kao i standardni Java thread-ovi





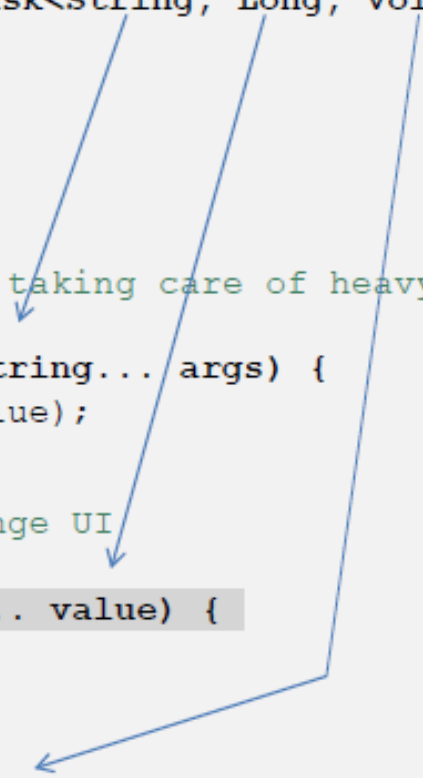
Android – AsyncTask

- ✿ AsyncTask je čistija alternativa bez Handlera i poruka
- ✿ AsyncTask predstavlja neku operaciju koja će se obaviti u pozadinskom thread-u i čiji će se rezultat predstaviti u UI thread-u
- ✿ AsyncTask ima 4 koraka
 - ❏ `onPreExecute`
 - ❏ `doInBackground`
 - ❏ `onProgressUpdate`
 - ❏ `onPostExecute`
- ✿ AsyncTask se uvek nasleđuje

Android – AsyncTask

- Parametrizuje se sa *Params*, *Progress* i *Result*

```
private class VerySlowTask extends AsyncTask<String, Long, Void> {  
  
    // Begin - can use UI thread here  
    protected void onPreExecute() {  
  
    }  
  
    // this is the SLOW background thread taking care of heavy tasks  
    // cannot directly change UI  
    protected Void doInBackground(final String... args) {  
        ... publishProgress((Long) someLongValue);  
    }  
  
    // periodic updates - it is OK to change UI  
    @Override  
    protected void onProgressUpdate(Long... value) {  
  
    }  
  
    // End - can use UI thread here  
    protected void onPostExecute(final Void unused) {  
  
    }  
  
}
```



Android – AsyncTask

- “Čistija” komunikacija sa UI thread-om i widget-ima
- Ne zahteva korišćenje Handler-a i thread-ova

3 Generic Types	4 Main States	1 Auxiliary Method
Params, Progress, Result	onPreExecute, doInBackground, onProgressUpdate onPostExecute.	publishProgress

AsyncTask's generic types

Params: the type of the parameters sent to the task upon execution.

Progress: the type of the progress units published during the background computation.

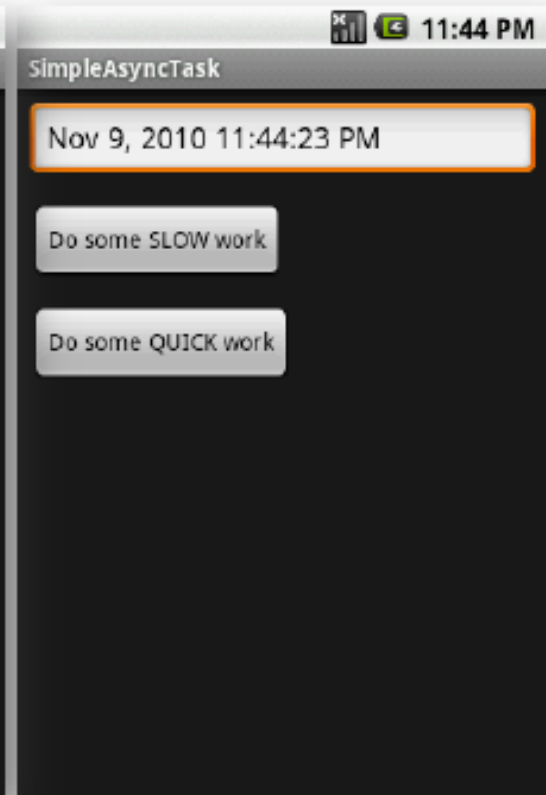
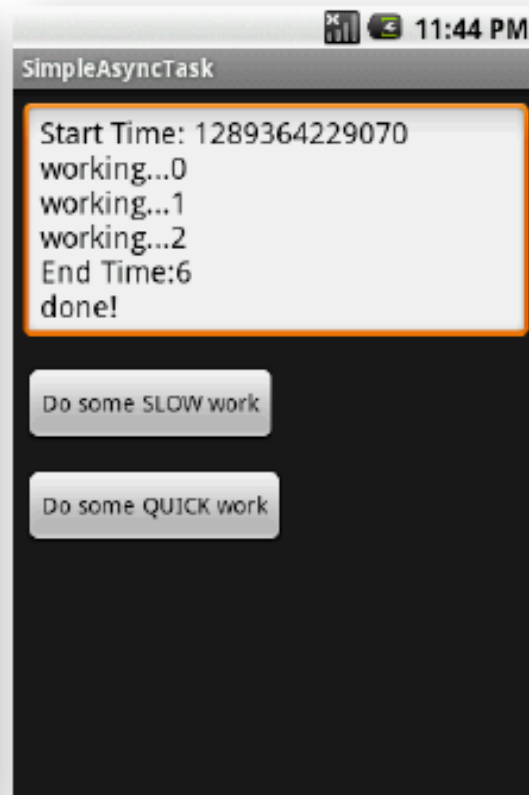
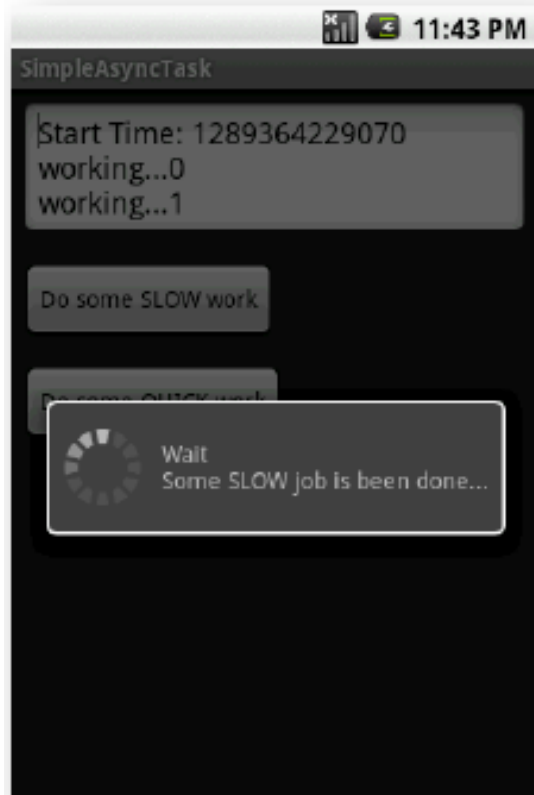
Result: the type of the result of the background computation.

Android – AsyncTask

- ✿ Tip koji se ne koristi može biti označen sa Void
- ✿ Upotreba callback metoda
- ✿ *onPreExecute()* – izvršava se na UI thread-u i koristi za inicijalno postavljanje UI
- ✿ *doInBackground(Params...)* – pozadinska operacija, u ovoj metodi može da se pozove *publishProgress(Progress..)*
- ✿ *onProgressUpdate(Progress...)* – update-uje UI u skladu sa progresom
- ✿ *onPostExecute(Result)* – prikazuje rezultat na UI widget-ima

Android – AsyncTask

- Glavni task poziva AsyncTask koji izračunava nešto i periodično update-uje UI (upisuje linije teksta) i menja cirkularni progress bar





Android – AsyncTask

```
public class Main extends Activity {
    Button btnSlowWork;
    Button btnQuickWork;
    EditText etMsg;
    Long startingMillis;

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        etMsg = (EditText) findViewById(R.id.EditText01);

        btnSlowWork = (Button) findViewById(R.id.Button01);
        // slow work...for example: delete all data from a database or get data from Internet
        this.btnSlowWork.setOnClickListener(new OnClickListener() {
            public void onClick(final View v) {
                new VerySlowTask().execute(); ←
            }
        });

        btnQuickWork = (Button) findViewById(R.id.Button02);
        // delete all data from database (when delete button is clicked)
        this.btnQuickWork.setOnClickListener(new OnClickListener() {
            public void onClick(final View v) {
                etMsg.setText((new Date()).toLocaleString());
            }
        });
    }
}
```

Android – AsyncTask

```
private class VerySlowTask extends AsyncTask <String, Long, Void> {

    private final ProgressDialog dialog = new ProgressDialog(Main.this);

    // can use UI thread here
    protected void onPreExecute() {
        startingMillis = System.currentTimeMillis();
        etMsg.setText("Start Time: " + startingMillis);
        this.dialog.setMessage("Wait\nSome SLOW job is being done...");
        this.dialog.show();
    }

    // automatically done on worker thread (separate from UI thread)
    protected Void doInBackground(final String... args) {
        try {
            // simulate here the slow activity
            for (Long i = 0L; i < 3L; i++) {
                Thread.sleep(2000);
                publishProgress((Long)i);
            }
        } catch (InterruptedException e) {
            Log.v("slow-job interrupted", e.getMessage())
        }
        return null;
    }
}
```

Android – AsyncTask

```
// periodic updates - it is OK to change UI
@Override
protected void onProgressUpdate(Long... value) {
    super.onProgressUpdate(value);

    etMsg.append("\nworking..." + value[0]);
}

// can use UI thread here
protected void onPostExecute(final Void unused) {

    if (this.dialog.isShowing()) {
        this.dialog.dismiss();
    }

    // cleaning-up, all done
    etMsg.append("\nEnd Time:"
        + (System.currentTimeMillis() - startingMillis) / 1000);
    etMsg.append("\ndone!");
}

} // AsyncTask

} // Main
```



Android – AsyncTaskLoader

- ✿ AsyncTask – problem kada se menja konfiguracija – rotacija ekrana portrait/landscape
- ✿ Uništavanje i rekreiranje aktivnosti
- ✿ AsyncTask ostaje vezan za zombie activity i njemu isporučuje podatke
- ✿ Novi activity može ponovo da startuje AsyncTask koji ponovo krene da učitava iste podatke
- ✿ Loše upravljanje memorijom
- ✿ *AsyncTaskLoader* implementira nit koja je vezana za šivotni ciklus aplikacije, ne restartuje se



Android – Kotlin korutine

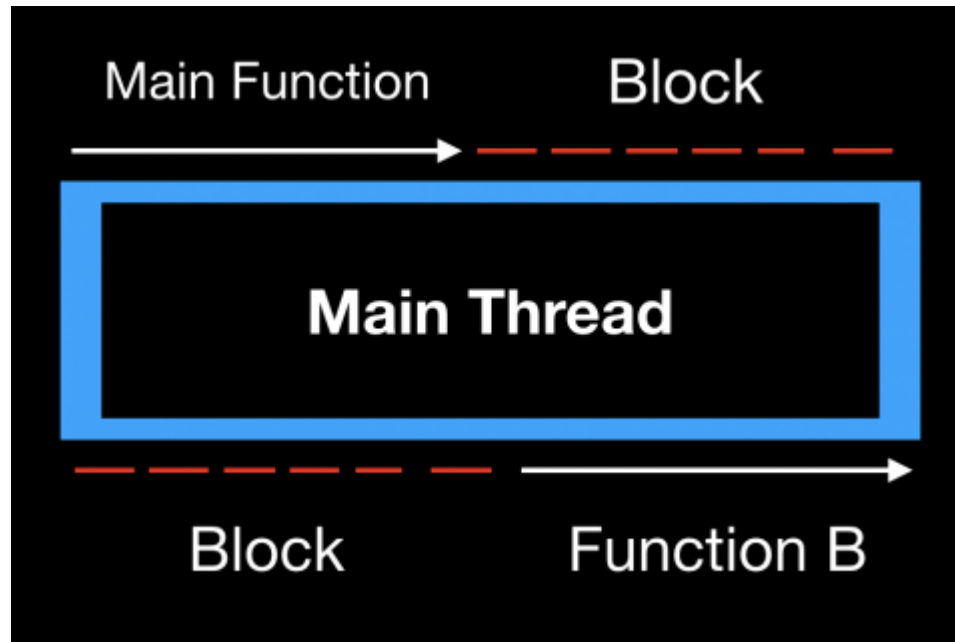
- ✚ Jednostavne za razumevanje
- ✚ Malo boilerplate koda
- ✚ Pišu se sekvencijalno
- ✚ Dependencies

```
implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-core:1.6.4'
implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.6.4'
```

- ✚ Korutine su „jeftinije“ za kreiranje pa se često nazivaju „lightweight threads“
- ✚ Korutine mogu da se suspenduju i nastave

Android – Kotlin korutine

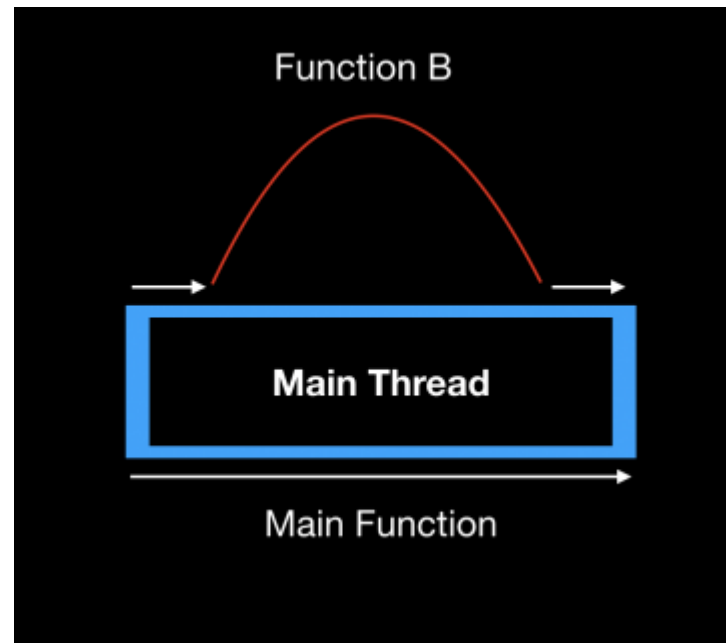
✿ Blokiranje vs. Suspendovanje



- ✿ Blokirajući poziv funkcije blokira nit u kojoj se izvršava dok se ne završi

Android – Kotlin korutine

- Prilikom suspendovanja funkcije trenutna nit je slobodna da izvršava drugi kod dok funkcija ne vrati vrednost



Android – Kotlin korutine

```
lifecycleScope.launch {  
    val originalBitmap = getOriginalBitmap(tutorial)  
    val snowFilterBitmap = loadSnowFilter(originalBitmap)  
    loadImage(snowFilterBitmap)  
}
```

- ❖ *lifecycleScope* je coroutine scope
- ❖ *Launch* je coroutine builder
- ❖ Programski kod se piše sekvencijalno
- ❖ *getOriginalBitmap* suspenduje korutinu i oslobađa nit da izvršava drugi kod dok se ne završi
- ❖ Kada se funkcija *getOriginalBitmap* završi korutina nastavlja sa izvršenjem

Android – Kotlin korutine

```
private suspend fun getOriginalBitmap(tutorial: Tutorial): Bitmap =  
    //2  
    withContext(Dispatchers.IO) {  
        //3  
        URL(tutorial.imageUrl).openStream().use {  
            return@withContext BitmapFactory.decodeStream(it)  
        }  
    }  
}
```

- ✿ Funkcija je markirana ključnom reči *suspend*
- ✿ *withContext(Dispatchers.IO)* izmešta dugotrajan posao u worker thread
- ✿ Dva načina za kreiranje korutine
 - ✦ *launch* – najčešće korišćen, kreira korutinu i vraća handle na *Job* objekat koji kasnije može da se koristi za otkazivanje
 - ✦ *async* – kada korutina treba da vrati odloženo neku vrednost



Android – Kotlin korutine

- ✿ *Coroutine scope* određuje koliko je korutina živa
- ✿ Postoje predefinisani scope
 - ✦ `lifecycleScope`
 - ✦ `viewModelScope`
 - ✦ `GlobalScope` – dok god postoji aplikacija
- ✿ Na primer, ako se koristi `lifecycleScope`, korutina se uništava kada se uništi fragment

Pitanja i komentari

